

SalePadel

Descripción del Pipeline de Datos

Entregable 4 · Arquitectura y Diseño de Sistemas 2026

Fecha de entrega: 08/06/2026

Pipeline 1 — Reporte Semanal

Agrega los datos de actividad de la semana (reservas, partidos, usuarios) y genera un reporte consolidado que el administrador puede visualizar y exportar.

PASO 1 — Origen de los datos

¿Qué datos?	Registros de las tablas: reserva (id_reserva, id_jugador, id_turno, estado), turno (id_turno, fecha, hora, precio, estado_turno), jugador (id_usuario, partidos_jugados, partidos_ganados, penalizaciones), EvaluacionJugador (id_evaluacion, id_evaluador, id_evaluado, id_reserva, puntaje, creada_en) y EvaluacionTurno (id_evaluacion, id_jugador, id_turno, puntaje, creada_en) . Formato: filas relacionales en PostgreSQL.
¿Desde dónde?	Base de datos PostgreSQL — tablas reserva, turno, jugador, EvaluacionJugador, EvaluacionTurno , cancha.
¿Con qué frecuencia?	Los datos fuente se actualizan continuamente durante la semana. El pipeline los lee una vez por semana al momento de ejecución.
¿Volumen estimado?	Estimado: 50–200 reservas/semana, 200–800 evaluaciones/semana. Orden de magnitud: cientos de filas por ejecución.

PASO 2 — Disparador

¿Cuándo se ejecuta?	Programado: cron job cada lunes a las 02:00 hs (zona horaria Argentina, UTC-3). También puede ejecutarse manualmente desde el panel de administración.
¿Quién lo dispara?	Scheduler interno del Backend Next.js (node-cron). En ejecución manual: request HTTP autenticado desde el frontend de administración.

PASO 3 — Procesamiento

¿Qué componente?	Backend Next.js — módulo Gestión Usuario y Gestión Turnos. ORM: Prisma sobre PostgreSQL.
------------------	--

Paso	Descripción
1 — Definir período	Calcular fecha_inicio (lunes 00:00) y fecha_fin (domingo 23:59) de la semana anterior.
2 — Extraer reservas	Consulta SQL con JOIN entre reserva, turno y cancha. Filtro: turno.fecha entre fecha_inicio y fecha_fin.

3 — Agregar por cancha	Calcular total de reservas, ingresos (suma de precios), promedio de ocupación y horas pico por cancha.
4 — Agregar por jugador	Calcular partidos jugados en el período, Calcular promedio de puntaje de EvaluacionJugador donde id_evaluado = id_jugador en el período. y penalizaciones generadas.
5 — Calcular métricas globales	Total de ingresos del período, cancha más demandada, hora pico global, porcentaje de cancelaciones.
6 — Construir objeto reporte	Consolidar resultados en estructura JSON: { periodo, canchas: [...], jugadores: [...], metricas_globales }.
7 — Persistir	Insertar registro en tabla reporte_semanal con el JSON consolidado y timestamp de generación.

¿Lógica de error?	Si la consulta falla (timeout o BD no disponible): el scheduler reintenta hasta 3 veces con backoff de 5 minutos. Si persiste, registra el error en log y notifica al administrador por mail. No se genera reporte en blanco para evitar datos inconsistentes.
--------------------------	--

PASO 4 — Salida

¿Qué se produce?	Registro en tabla reporte_semanal: { id, periodo_inicio, periodo_fin, datos_json, fecha_generacion}. En exportación: archivo PDF o Excel generado bajo demanda.
¿Dónde se guarda?	PostgreSQL — tabla reporte_semanal. El archivo exportado se genera en memoria y se transfiere directamente al navegador (no se persiste en disco).
¿Quién consume?	Frontend: Vista de reportes (administrador), accediendo a GET /api/reportes/:id.

PASO 5 — Decisión Arquitectónica

¿Por qué Batch?	Los datos de reportes son históricos — no requieren tiempo real. Ejecutar el agregado una vez por semana en horario de baja carga (02:00 hs) evita impacto en performance del sistema durante operación normal. Los administradores consultan reportes en tiempo real, pero ya fueron generados anteriormente.
Trade-offs aceptados	El reporte refleja datos hasta el cierre del período anterior — no incluye actividad del día actual. La exportación a PDF/Excel se hace on-demand, no se pre-genera.
ADR relacionado	Ver ADR-001 (Arquitectura Monolito Modular) — el módulo de reportes es independiente, no comparte lógica con otros componentes, solo consulta datos.

Pipeline 2 — Notificaciones por Eventos de Dominio

Detecta cambios de estado críticos en el dominio (lobby confirmado, turno próximo, turno finalizado, solicitud rechazada) y despacha correos electrónicos a los jugadores afectados.

PASO 1 — Origen de los datos

¿Qué datos?	Eventos de dominio generados por los módulos matchmaking y booking: { tipo_evento, id_entidad, timestamp, usuarios_destinatarios[], datos_contexto }. Los datos de contexto incluyen nombre del jugador, fecha/hora del turno, cancha y estado resultante.
¿Desde dónde?	Eventos internos emitidos en la capa application de los módulos matchmaking (lobby confirmado, solicitud rechazada) y booking (turno próximo, turno finalizado). No hay broker externo: los eventos se emiten in-process mediante un EventEmitter interno y se persisten en tabla notificacion.
¿Con qué frecuencia?	En tiempo real, ante cada evento de dominio relevante. Disparadores identificados: (1) lobby pasa a estado Confirmado, (2) 2 horas antes del inicio del turno, (3) turno pasa a Finalizado, (4) solicitud de lobby es rechazada.
¿Volumen estimado?	Estimado: 20–80 notificaciones/día. Pico: ~16 mails simultáneos al completarse varios lobbies en horario de alta demanda.

PASO 2 — Disparador

¿Cuándo se ejecuta?	Event-driven: inmediatamente al ocurrir el evento de dominio. Excepción: el recordatorio de 2h antes es disparado por cron job cada 15 minutos que consulta turnos cuya hora de inicio esté entre ahora y ahora+2h.
¿Quién lo dispara?	Los casos de uso de la capa application emiten el evento al completar la operación. El scheduler (node-cron cada 15 min) dispara los recordatorios de turnos próximos.

PASO 3 — Procesamiento

¿Qué componente?	Backend Next.js — módulo notifications (bounded context independiente). Tecnología: Nodemailer + SMTP externo (SendGrid o similar).
------------------	---

Paso	Descripción
1 — Recibir evento	El módulo notifications recibe el evento de dominio con el tipo y el contexto. Persiste una fila en tabla notificacion con estado='pendiente'.
2 — Identificar destinatarios	Según tipo_evento, consulta los usuarios afectados: para lobby_confirmado son los 4 jugadores; para solicitud_rechazada es solo el solicitante.
3 — Recuperar datos de contacto	Consulta correo_electronico de cada destinatario en tabla usuario.
4 — Renderizar mensaje	Selecciona template según tipo_evento e inyecta datos de contexto (nombre jugador, cancha, fecha, hora). Templates: confirmacion_lobby, recordatorio_turno, turno_finalizado, solicitud_rechazada.
5 — Enviar via SMTP	Llama al cliente SMTP externo (Nodemailer). Registra timestamp de envío.
6 — Actualizar estado	Actualiza fila en tabla notificacion: estado='enviada' o estado='fallida' según respuesta del SMTP.

¿Lógica de error?	Si el SMTP falla: reintento automático con backoff exponencial (1 min, 5 min, 15 min). Máximo 3 reintentos. Si persiste, estado queda 'fallida' en la tabla notificacion para auditoría . El fallo de notificación nunca bloquea el flujo principal de negocio
-------------------	--

PASO 4 — Salida

¿Qué se produce?	Correos electrónicos entregados a los jugadores. Registro actualizado en tabla notificacion con estado final y timestamp.
¿Dónde se guarda?	PostgreSQL — tabla notificacion ({ id, tipo, id_destinatario, estado, intentos, creada_en, enviada_en }). El mail es entregado a la casilla del usuario final via API SMTP externa.
¿Quién consume?	Usuario final (jugador): recibe el correo en su bandeja. Módulo de auditoría/reporting: puede consultar historial de notificaciones enviadas.

PASO 5 — Decisión Arquitectónica

¿Por qué Event-driven?	Las notificaciones son una consecuencia de eventos de negocio, no de horarios. Un lobby que se confirma a las 14:37 debe notificar en ese momento, no en la próxima ventana de batch. El modelo event-driven mantiene el SRP: el módulo matchmaking no sabe nada de mails — solo emite un evento.
¿Por qué SMTP externo y no in-app?	el correo electrónico es el canal más universal. No requiere que el usuario esté conectado en el momento del evento.

Trade-offs aceptados	Dependencia de un servicio SMTP externo (punto de fallo externo). Entrega de mail no garantizada en tiempo real si el proveedor tiene latencia. La tabla notificación actúa como dead-letter queue básica, sin la robustez de un broker como RabbitMQ o Redis Streams.
ADR relacionado	Ver ADR-003 (Comunicación entre componentes) — justifica el uso de eventos in-process sobre un broker externo para este tamaño de proyecto.

Pipeline 3 — Integración API Clima (ETL on-demand)

Extrae datos meteorológicos de una API externa para la fecha y ubicación de un turno específico, los transforma al formato interno del sistema, y los entrega al usuario que consulta el pronóstico de su partido.

PASO 1 — Origen de los datos

¿Qué datos?	Entrada: id_turno. El sistema recupera de la BD: fecha, hora y ubicación (latitud/longitud del complejo). La API externa devuelve: { temperatura, descripcion_clima, humedad, velocidad_viento, probabilidad_lluvia } para el bloque horario consultado.
¿Desde dónde?	(a) Tabla turno en PostgreSQL — para recuperar fecha/hora. (b) Configuración del sistema — para recuperar ubicación fija del complejo. (c) API de Clima externa via HTTP GET.
¿Con qué frecuencia?	On-demand: únicamente cuando el usuario presiona 'Ver pronóstico' en el detalle de un turno. No hay polling. Antes de llamar a la API externa, el sistema verifica si ya existe un dato vigente en caché.
¿Volumen estimado?	Estimado: 50–200 consultas/día. Una llamada HTTP por consulta de usuario.

PASO 2 — Disparador

¿Cuándo se ejecuta?	En tiempo real, ante acción explícita del usuario. El trigger es un request HTTP GET a /api/clima?turnold={id} desde el frontend.
¿Quién lo dispara?	El usuario (jugador) desde la vista de detalle de turno. El frontend llama al endpoint del backend, que actúa como proxy/Facade hacia la API externa.

PASO 3 — Procesamiento

¿Qué componente?	Backend Next.js — módulo integrations, componente ControladorClima (Facade). Tecnología: fetch nativo de Node.js hacia la API externa.
------------------	--

Paso	Descripción
1 — Validar turno	Verificar que el id_turno existe y que el usuario tiene acceso a él. Recuperar fecha y hora del turno desde la BD.
2 — Recuperar datos del turno	El sistema consulta PostgreSQL para obtener la fecha y hora del turno. También obtiene la ubicación del complejo deportivo desde una configuración fija.
3 — Construir clave de caché	Se genera una clave única para identificar el pronóstico solicitado. La clave puede construirse usando la fecha, la hora o bloque horario, la latitud y la longitud. Por ejemplo: <code>clima:{lat}:{lon}:{fecha}:{horaBloque}</code> (Todavía no definido).
4 — Consultar caché temporal	Antes de consultar la API externa, el sistema verifica si existe un pronóstico cacheado vigente para esa clave. Si existe y no expiró, se devuelve ese dato directamente al frontend
5 — Construir request a la API externa	Si no hay dato en caché o el dato está vencido, el backend construye la URL de consulta hacia la API externa, incluyendo latitud, longitud, fecha y hora del turno.
6 — Llamar API externa	HTTP GET a la API de Clima. Timeout configurado en 3000ms
7 — Validar respuesta	Verificar que la respuesta HTTP sea 200 y que el body contenga los campos esperados. Detectar si la fecha del turno supera el límite de predicción de la API (ej. más de 7 días).
8 — Transformar formato	Mapear campos de la API externa al DTO interno: { temperatura_celsius, descripcion, humedad_porcentaje, viento_kmh, probabilidad_lluvia_porcentaje, fecha_consulta }.
9 — Guardar en caché	El resultado transformado se almacena temporalmente en caché con un TTL. El dato no se considera información permanente de negocio, sino una optimización para reducir llamadas repetidas a la API externa.
10 — Retornar al cliente	El Facade devuelve el DTO al Route Handler, que lo envuelve en { data, error, meta } y responde al frontend.

¿Lógica de error?	Si la API externa no responde (timeout): retornar { data: null, error: 'El pronóstico no está disponible en este momento' } — nunca propagar el error crudo al frontend. Si la fecha supera el horizonte de predicción: retornar { data: null, error: 'El pronóstico aún no está disponible para esta fecha' } (CDU 9 — excepción q). No hay reintentos automáticos: la consulta es interactiva y el usuario puede reintentar manualmente.
-------------------	--

PASO 4 — Salida

¿Qué se produce?	Se produce un DTO de clima enriquecido para ser consumido por el frontend. El formato interno puede ser: { temperatura_celsius, descripcion, humedad_porcentaje, viento_kmh, probabilidad_lluvia_porcentaje, fecha_consulta }.
------------------	--

¿Dónde se guarda?	El resultado se guarda temporalmente en una caché.
¿Quién consume?	Frontend: Vista de detalle de turno — muestra el pronóstico al usuario en tiempo real.

PASO 5 — Decisión Arquitectónica

¿Por qué ETL on-demand y no Batch?	Se elige un enfoque ETL on-demand con caché temporal porque el sistema necesita integrar datos provenientes de una API externa, transformarlos al formato interno y entregarlos cuando el usuario los solicita. No se elige procesamiento batch porque no todos los turnos serán consultados y porque consultar pronósticos de forma anticipada podría generar datos obsoletos o llamadas innecesarias. La caché temporal se incorpora para evitar repetir consultas idénticas, reducir latencia y proteger el límite de uso de la API gratuita.
¿Por qué Facade en el backend y no llamada directa desde el frontend?	Exponer la API key de clima en el frontend es un riesgo de seguridad . El Facade en el backend centraliza el manejo de errores, el timeout y la transformación de formato, cumpliendo OCP: si se cambia el proveedor de clima, solo cambia el Facade.
Trade-offs aceptados	Latencia acumulada: el usuario espera la ida al backend + la ida a la API externa + la vuelta. Si la API externa es lenta, la experiencia se degrada (RNF5 — mitigado con timeout de 3000ms y mensaje de error claro). Sin caché: si 50 usuarios consultan el clima del mismo turno en el mismo minuto, se generan 50 llamadas a la API externa.
ADR relacionado	Ver ADR-001 (Arquitectura Modular) — el ControladorClima vive en el módulo integrations como Facade, aislando al resto del sistema de la API externa.

Pipeline 4 — Cálculo de Métricas de Jugador

Recalcula la reputación promedio, penalizaciones y categorías derivadas (Jugador Destacado, Usuario poco confiable) de un jugador a partir de sus evaluaciones y comportamiento histórico.

PASO 1 — Origen de los datos

¿Qué datos?	Tabla evaluacionJugador:{ id_evaluacion, id_evaluador, id_evaluado, id_reserva, puntaje, creada_en }. Tabla evaluacionTurno:{ id_evaluacion, id_jugador, id_turno, puntaje, creada_en }. Tabla reserva: estado de reservas por jugador. Tabla jugador: { penalizaciones, reputacion_promedio, evaluaciones_recibidas }. Período relevante para inferencias: últimas 4 semanas (para horarios de alta demanda) y último mes (para usuario poco confiable).
¿Desde dónde?	PostgreSQL — tablas evaluacion, reserva, jugador. Evento de dominio que indica qué jugador fue evaluado o qué reserva fue cancelada.
¿Con qué frecuencia?	Modo event-driven: al completarse una evaluación o al registrarse una cancelación tardía (< 12h). Modo batch: las inferencias de categoría (Destacado, Poco Confiable) se recalculan diariamente a las 03:00 hs para todos los jugadores activos.

PASO 2 — Disparador

¿Cuándo se ejecuta?	Dos modos: (1) Event-driven — inmediatamente al recibir evento evaluacion_registrada o cancelacion_tardia_registrada. (2) Batch — cron job diario a las 03:00 hs para recalcular inferencias.
¿Quién lo dispara?	(1) Módulo de turnos y evaluaciones emiten eventos de dominio. (2) Scheduler interno para el batch nocturno.

PASO 3 — Procesamiento

¿Qué componente?	Backend Next.js — módulo turnos o feedback.
------------------	---

Paso	Descripción
1 — Recibir evento / trigger	Identificar el id_jugador afectado. En modo batch: iterar sobre lista de jugadores activos.

2 — Recalcular reputación	SELECT AVG(puntaje) FROM evaluacionJugador WHERE id_evaluado = ? → actualizar jugador.reputacion_promedio y jugador.evaluaciones_recibidas. SELECT AVG(puntaje) FROM evaluacionTurno WHERE id_jugador = ? → disponible para estadísticas del turno o reportes, no afecta directamente las inferencias de Destacado ni Poco Confiable.
3 — Recalcular penalizaciones	Contar cancelaciones con menos de 12h de preaviso en el mes actual → actualizar jugador.penalizaciones.
4 — Evaluar inferencia: Jugador Destacado	Si reputacion_promedio > 4.5 Y evaluaciones_recibidas >= 5 → marcar jugador.es_destacado = true. Si ya no cumple → false.
5 — Evaluar inferencia: Usuario poco confiable	Si penalizaciones_mes > 3 → marcar jugador.es_poco_confiable = true Y bloquear temporalmente apertura de lobbies. Duración del bloqueo: definir con el grupo (recomendado: 30 días).
6 — Evaluar inferencia: Horario de Alta Demanda	Batch semanal adicional: para cada franja horaria, calcular tasa de ocupación en las últimas 4 semanas. Si > 90% → marcar franja como alta_demanda para precios dinámicos (RF17).
7 — Persistir cambios	UPDATE jugador SET reputacion_promedio, evaluaciones_recibidas, penalizaciones, es_destacado, es_poco_confiable WHERE id_usuario = ?

¿Lógica de error?	Si falla el recálculo event-driven: el error se loggea pero no revierte la operación original (la evaluación ya fue guardada). El batch del día siguiente corregirá el estado. Si falla el batch nocturno: reintento al día siguiente — las métricas son tolerantes a una latencia de 24h.
--------------------------	--

PASO 4 — Salida

¿Qué se produce?	Campos actualizados en tabla jugador: reputacion_promedio, evaluaciones_recibidas, penalizaciones, es_destacado, es_poco_confiable. Calculados desde tabla evaluacionJugador. Para alta demanda: campo actualizado en tabla turno. La tablas evaluacionTurno se persiste para uso en reportes y estadísticas del complejo.
¿Dónde se guarda?	PostgreSQL — tabla jugador (métricas personales). Tabla precio_franja (si se implementan precios dinámicos por alta demanda).
¿Quién consume?	Frontend: Vista de perfil deportivo del jugador. Módulo matchmaking: valida es_poco_confiable antes de permitir crear un lobby. Módulo booking: usa precio dinámico si la franja es alta_demanda.

PASO 5 — Decisión Arquitectónica

¿Por qué híbrido Event-driven + Batch?	La reputación post-evaluación se recalcula event-driven porque el jugador evaluado merece ver su puntaje actualizado de forma inmediata. Las inferencias categóricas (Destacado, Poco Confiable) son menos urgentes y requieren analizar ventanas de tiempo (último mes, últimas 4 semanas) — el batch nocturno evita recalcular estas ventanas en cada operación.
¿Por qué no calcular on-the-fly?	Calcular AVG(evaluaciones) en cada consulta de perfil viola < 200ms bajo carga) cuando un jugador tiene cientos de partidos. Persistir el promedio y actualizarlo incrementalmente es el trade-off correcto.
Trade-offs aceptados	Las inferencias pueden tener hasta 24h de latencia respecto a la realidad (un jugador que supera 3 cancelaciones a las 22:00 podría abrir un lobby hasta las 03:00 del día siguiente). La reputación post-evaluación es eventualmente consistente si el event-driven falla — el batch nocturno actúa como reconciliador.
ADR relacionado	Ver ADR-001 (Arquitectura Modular) — MetricsJugadorService vive en el módulo engagement, no en booking. La comunicación entre módulos se hace via eventos de dominio, no importando servicios entre bounded contexts. Ver también ADR-003 (Comunicación entre componentes) — los eventos de dominio evaluacion_registrada y cancelacion_tardia_registrada que disparan este pipeline son emitidos mediante EventEmitter entre módulos, según la estrategia definida en ADR-003.